

Phase 1: AVL Foundations — BST, Height, and Balance Factor

1. Goal of Phase 1

In this phase, we learn the basic ideas behind AVL trees.

We are **not** learning rotations yet. Rotations come later.

The goal of Phase 1 is to understand:

- What a BST is
- Why a normal BST can become slow
- What an AVL tree is
- What height means
- What balance factor means
- Which balance factors are valid
- How to trace height and balance factor by hand
- Why each AVL node stores height
- How simple helper functions work in C
- Common beginner mistakes

The most important idea is this:

An AVL tree is a Binary Search Tree where every node must stay balanced.

2. What Is a Binary Search Tree?

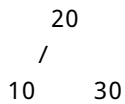
A **Binary Search Tree**, or **BST**, is a binary tree that follows this rule:

`left side < node < right side`

That means:

- All values on the left side of a node are smaller than the node.
- All values on the right side of a node are greater than the node.

Example:



This is a valid BST because:

$10 < 20 < 30$

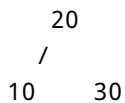
Node **10** is smaller than **20**, so it is on the left.

Node **30** is greater than **20**, so it is on the right.

3. Why BSTs Are Useful

A BST helps us search by making decisions.

Example: searching for **30**



Steps:

Start at 20.
30 is greater than 20, so go right.
Found 30.

This is useful because we do not need to check every node if the tree is well-shaped.

A good BST can search quickly because each step removes part of the tree from consideration.

4. The Problem with a Normal BST

A normal BST does **not** automatically stay balanced.

If we insert sorted values like this:

10, 20, 30, 40

The BST may become:

```
10
  20
    30
      40
```

This is still a valid BST because every right child is greater than its parent.

But the shape is bad.

It looks like a linked list.

Searching for 40 takes several steps:

10 -> 20 -> 30 -> 40

So the search becomes slow.

In the worst case, a normal BST can take:

$O(N)$

That means if there are many nodes, we may need to walk through many of them.

5. What Is an AVL Tree?

An **AVL tree** is a special kind of Binary Search Tree.

It follows the normal BST rule:

left < node < right

But it also keeps itself balanced.

In simple words:

An AVL tree is a BST that does not allow one side of a node to become much taller than the other side.

Think of each node as asking:

```
Is my left side too tall?  
Is my right side too tall?  
Am I still balanced?
```

If a node becomes too unbalanced, the AVL tree must fix it.

In Phase 1, we only learn how to **detect** imbalance.

The fixing part is done using **rotations**, which are taught in Phase 2.

6. Why AVL Trees Exist

AVL trees exist to solve the main weakness of normal BSTs.

A normal BST can become tall and slow.

An AVL tree tries to stay short and balanced.

Because of this, AVL tree operations such as search, insertion, and deletion can stay efficient.

The key idea is:

```
Shorter tree = fewer steps to search  
Taller tree = more steps to search
```

AVL trees protect us from badly shaped BSTs.

7. What Is Height?

In this curriculum, height is counted using this rule:

```
Missing child / NULL child height = 0
Leaf node height = 1
```

A **leaf node** is a node with no children.

Example:

```
10
```

Node **10** has no left child and no right child.

So:

```
left height = 0
right height = 0
height of 10 = 1
```

The formula for height is:

```
height(node) = 1 + max(height(left child), height(right child))
```

This means:

1. Find the height of the left child.
2. Find the height of the right child.
3. Take the bigger one.
4. Add 1 for the current node.

8. Height Example 1: Single Node

Tree:

```
10
```

Node **10** has no children.

```
left height = 0
right height = 0
```

```
height = 1 + max(0, 0)
height = 1
```

So the height of node 10 is:

1

9. Height Example 2: One Child

Tree:

```
  20
 /
10
```

Node 10 is a leaf.

```
height(10) = 1
```

Node 20 has:

```
left height = 1
right height = 0
```

So:

```
height(20) = 1 + max(1, 0)
height(20) = 2
```

So the height of node 20 is:

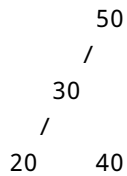
2

10. Important Rule About Height

Height does **not** mean total number of nodes.

Height means the longest downward path from that node.

Example:



The left subtree of 50 contains three nodes:

30, 20, 40

But the height of that subtree is not 3.

The height is 2, because the longest downward path is:

30 -> 20

or:

30 -> 40

That path has height 2.

So remember:

Height = longest downward path
Height $\neq$ total number of nodes

11. What Is Balance Factor?

The **Balance Factor** tells us whether a node is balanced.

It is often written as:

BF

The formula is:

Balance Factor = height(left subtree) - height(right subtree)

Short form:

BF = left height - right height

This is the main formula in Phase 1.

12. Meaning of Balance Factor Values

BF = 0

left height = right height

The node is perfectly balanced.

Example:

```
    20
   /
  10  30
```

For node 20:

```
left height = 1
right height = 1
BF = 1 - 1 = 0
```

BF = +1

```
left side is taller by 1
```

The node is still balanced.

Example:

```
  20
 /
10
```

For node 20:

```
left height = 1
right height = 0
BF = 1 - 0 = 1
```

This is valid.

BF = -1

```
right side is taller by 1
```

The node is still balanced.

Example:

```
20
  30
```

For node 20:

```
left height = 0
right height = 1
BF = 0 - 1 = -1
```

This is valid.

BF = +2

```
left side is taller by 2
```

This is not valid in an AVL tree.

The node is imbalanced.

BF = -2

```
right side is taller by 2
```

This is not valid in an AVL tree.

The node is imbalanced.

13. Valid and Invalid Balance Factors

In an AVL tree, each node must have one of these balance factors:

Balance Factor	Meaning	AVL Status
-1	Right side is taller by 1	Valid
0	Both sides have same height	Valid
+1	Left side is taller by 1	Valid

These are invalid:

Balance Factor	Meaning	AVL Status
+2	Left side is too tall	Invalid

Balance Factor	Meaning	AVL Status
-2	Right side is too tall	Invalid
+3, -3, etc.	Difference is even bigger	Invalid

Simple rule:

Allowed BF values: -1, 0, +1
Not allowed: anything else

Another way to say it:

If absolute value of BF is greater than 1, the node is imbalanced.

Examples:

BF = 1 -> valid
BF = 0 -> valid
BF = -1 -> valid
BF = 2 -> invalid
BF = -2 -> invalid

14. Balanced Tree Example

Tree:

```

      20
     /
    10  30

```

Let us calculate every node.

Node 10

Node 10 has no children.

left height = 0
right height = 0

```
height = 1
BF = 0 - 0 = 0
```

Node 10 is balanced.

Node 30

Node 30 has no children.

```
left height = 0
right height = 0
height = 1
BF = 0 - 0 = 0
```

Node 30 is balanced.

Node 20

Node 20 has two children.

```
left height = 1
right height = 1
height = 1 + max(1, 1)
height = 2
BF = 1 - 1 = 0
```

Node 20 is balanced.

So the whole tree is balanced.

15. Imbalanced Tree Example

Tree:

```
    30
   /
  20
```

/
10

We should calculate from bottom to top.

This is important because parent height depends on child height.

Step 1: Node 10

Node 10 is a leaf.

```
left height = 0
right height = 0
height = 1
BF = 0 - 0 = 0
```

Node 10 is balanced.

Step 2: Node 20

Node 20 has left child 10.

```
left height = 1
right height = 0
height = 1 + max(1, 0)
height = 2
BF = 1 - 0 = 1
```

Node 20 is still balanced.

Step 3: Node 30

Node 30 has left subtree height 2 and no right child.

```
left height = 2
right height = 0
height = 1 + max(2, 0)
```

```
height = 3
BF = 2 - 0 = 2
```

Node 30 is imbalanced because:

```
BF = 2
```

2 is not allowed in an AVL tree.

So this tree needs to be fixed later by a rotation.

16. Why We Trace from Bottom to Top

When calculating height and balance factor, always start from the leaves.

This is because a parent node needs to know the height of its children first.

Example:

```
    30
   /
  20
 /
10
```

You cannot correctly calculate the height of 30 until you know the height of 20.

You cannot correctly calculate the height of 20 until you know the height of 10.

So the correct order is:

```
10 -> 20 -> 30
```

This is called bottom-up tracing.

17. AVL Node Structure in C

A normal BST node usually stores:

```
data
left pointer
right pointer
```

An AVL node stores one extra thing:

```
height
```

C structure:

```
typedef struct Node {
    int data;
    int height;
    struct Node *left;
    struct Node *right;
} Node;
```

Meaning of each field:

Field	Meaning
data	The value stored in the node
height	Height of the subtree rooted at this node
left	Pointer to the left child
right	Pointer to the right child

18. Why AVL Nodes Store Height

AVL trees often need to calculate balance factor.

Balance factor needs height:

```
BF = left height - right height
```

If height is stored inside each node, the program can read it quickly.

Reading stored height takes:

$O(1)$

That means constant time.

If height was not stored, the program might need to scan down the subtree many times to calculate height.

That would make AVL operations slower.

So AVL nodes store height to make balance checks fast.

19. Helper Function: getHeight

A helper function makes code easier to read.

```
int getHeight(Node *n) {  
    if (n == NULL) {  
        return 0;  
    }  
    return n->height;  
}
```

This function means:

If the node is NULL, return 0.
Otherwise, return the node's stored height.

Why return `0` for `NULL` ?

Because a missing child has height `0` in this curriculum.

Example:

```
    20  
   /  
  10
```

Node `20` has no right child.

So:

```
height(right child) = 0
```

The `getHeight` function handles that safely.

20. Helper Function: `getBalanceFactor`

The balance factor helper function uses `getHeight`.

```
int getBalanceFactor(Node *n) {  
    if (n == NULL) {  
        return 0;  
    }  
  
    return getHeight(n->left) - getHeight(n->right);  
}
```

This follows the formula:

```
BF = left height - right height
```

If `n` is `NULL`, the function returns `0`.

This prevents the program from trying to access a missing node.

21. Updating Height

When a node gets a new child, its height may change.

So we need to update the node's height.

Example helper function:

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}  
  
void updateHeight(Node *n) {  
    if (n != NULL) {
```

```

        n->height = 1 + max(getHeight(n->left), getHeight(n->right));
    }
}

```

This uses the formula:

```
height = 1 + max(left height, right height)
```

Heights should be updated from bottom to top.

That means update child heights before parent heights.

22. Complete Mini Program

This program manually creates an imbalanced tree:

```

      30
     /
    20
   /
  10

```

Then it prints each node's height and balance factor.

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    int height;
    struct Node *left;
    struct Node *right;
} Node;

int max(int a, int b) {
    return (a > b) ? a : b;
}

int getHeight(Node *n) {
    if (n == NULL) {
        return 0;
    }
}

```

```

        return n->height;
    }

    int getBalanceFactor(Node *n) {
        if (n == NULL) {
            return 0;
        }
        return getHeight(n->left) - getHeight(n->right);
    }

    Node *createNode(int value) {
        Node *n = (Node *)malloc(sizeof(Node));
        n->data = value;
        n->height = 1;
        n->left = NULL;
        n->right = NULL;
        return n;
    }

    void updateHeight(Node *n) {
        if (n != NULL) {
            n->height = 1 + max(getHeight(n->left), getHeight(n->right));
        }
    }

    void printNodeStatus(Node *n) {
        if (n == NULL) {
            return;
        }

        printf("Node %d: height=%d, balance factor=%d\n",
            n->data, n->height, getBalanceFactor(n));
    }

    int main(void) {
        Node *root = createNode(30);
        root->left = createNode(20);
        root->left->left = createNode(10);

        // Update from bottom to top.
        updateHeight(root->left);
        updateHeight(root);

        printNodeStatus(root->left->left);
        printNodeStatus(root->left);
        printNodeStatus(root);
    }

```

```
    return 0;  
}
```

23. Program Dry Run

The program creates this tree:

```
    30  
   /  
  20  
 /  
10
```

At first, every new node starts with height **1** because every new node is created as a leaf.

Then we connect the nodes:

```
30 gets left child 20  
20 gets left child 10
```

Now we update heights from bottom to top.

Update node 20

Node **20** has:

```
left child = 10  
right child = NULL
```

So:

```
left height = 1  
right height = 0  
height(20) = 1 + max(1, 0)  
height(20) = 2  
BF(20) = 1 - 0  
BF(20) = 1
```

Update node 30

Node 30 has:

```
left child = 20  
right child = NULL
```

So:

```
left height = 2  
right height = 0  
height(30) = 1 + max(2, 0)  
height(30) = 3  
BF(30) = 2 - 0  
BF(30) = 2
```

Output

The output should be:

```
Node 10: height=1, balance factor=0  
Node 20: height=2, balance factor=1  
Node 30: height=3, balance factor=2
```

Node 30 is imbalanced because its balance factor is 2.

24. Time Complexity in Phase 1

Reading stored height

If each node stores its height, then reading the height is fast.

```
getHeight(node) = O(1)
```

That means constant time.

Calculating one balance factor

Balance factor uses two height values:

```
BF = left height - right height
```

Since each height is read in constant time:

```
getBalanceFactor(node) = O(1)
```

Checking every node

If we check the balance factor of every node in a tree with N nodes, that takes:

```
O(N)
```

because every node must be visited once.

Recursive space

If we use recursion to check the whole tree, the extra space depends on the height of the tree.

```
Space = O(H)
```

where H is the height of the tree.

25. Common Beginner Mistakes

Mistake 1: Counting total nodes instead of height

Wrong idea:

```
This subtree has 3 nodes, so its height is 3.
```

Correct idea:

Height means longest downward path, not total nodes.

How to avoid it:

Start from the leaf nodes and trace upward.

Mistake 2: Thinking $BF = 2$ is okay temporarily

Wrong idea:

$BF = 2$ is only slightly unbalanced, so maybe it is okay.

Correct idea:

In an AVL tree, $BF = 2$ or $BF = -2$ is invalid.

AVL nodes must have BF values only from this set:

-1, 0, +1

Mistake 3: Forgetting that NULL height is 0

Wrong idea:

Try to read height from a missing child.

Correct idea:

A NULL child has height 0.

That is why `getHeight(NULL)` returns `0`.

Mistake 4: Updating parent height before child height

Wrong idea:

Update 30 before updating 20.

Correct idea:

Update children first, then parents.

The parent depends on the child heights.

Mistake 5: Forgetting that AVL is still a BST

Wrong idea:

AVL trees are completely different from BSTs.

Correct idea:

An AVL tree is a BST with an extra balancing rule.

It still follows:

$\text{left} < \text{node} < \text{right}$

26. Phase 1 Key Formulas

BST Rule

$\text{left subtree values} < \text{node value} < \text{right subtree values}$

Height Formula

$\text{height}(\text{node}) = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$

Balance Factor Formula

```
BF = height(left subtree) - height(right subtree)
```

Valid AVL Balance Rule

```
BF must be -1, 0, or +1
```

Invalid AVL Balance Rule

```
If BF is +2 or -2, the node is imbalanced.
```

27. Phase 1 Mental Model

Imagine every node checks its two sides.

It asks:

```
How tall is my left side?  
How tall is my right side?  
What is left height - right height?
```

Then it decides:

```
BF = -1 -> okay  
BF = 0  -> okay  
BF = +1 -> okay  
BF = +2 or -2 -> not okay
```

This is the heart of AVL balancing.

28. Step-by-Step Method to Check an AVL Tree

When you are given a tree, use this method:

Step 1

Start at the leaf nodes.

Leaf height is always:

1

Step 2

Move upward one node at a time.

For each node, calculate:

```
height = 1 + max(left height, right height)
```

Step 3

Calculate balance factor:

```
BF = left height - right height
```

Step 4

Check whether BF is valid:

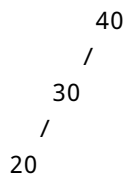
```
-1, 0, +1 -> valid  
+2, -2, etc. -> invalid
```

Step 5

If any node is invalid, the tree is not AVL balanced.

29. Worked Practice Example

Tree:



Start from the bottom.

Node 20

```
left height = 0
right height = 0
height = 1
BF = 0 - 0 = 0
```

Node 20 is balanced.

Node 30

```
left height = 1
right height = 0
height = 2
BF = 1 - 0 = 1
```

Node 30 is balanced.

Node 40

```
left height = 2
right height = 0
height = 3
BF = 2 - 0 = 2
```

Node 40 is imbalanced.

So the answers are:

```
BF(20) = 0
BF(30) = 1
BF(40) = 2
Node 40 is imbalanced.
```

30. Another Practice Example

Tree:

```
      50
     /
    30   70
   /
  20
```

Try to calculate the balance factor of each node.

Node 20

```
left height = 0
right height = 0
height = 1
BF = 0
```

Node 30

```
left height = 1
right height = 0
height = 2
BF = 1
```

Node 70

```
left height = 0
right height = 0
```

```
height = 1
BF = 0
```

Node 50

```
left height = 2
right height = 1
height = 3
BF = 2 - 1 = 1
```

All balance factors are valid:

```
20 -> BF 0
30 -> BF 1
70 -> BF 0
50 -> BF 1
```

So this tree is AVL balanced.

31. Quick Review Questions

Question 1

What is a BST?

Answer:

```
A Binary Search Tree is a tree where left values are smaller than the node and
right values are greater than the node.
```

Question 2

What extra rule does an AVL tree follow?

Answer:

Every node must stay balanced, meaning its balance factor must be -1, 0, or +1.

Question 3

What is the height of a NULL child?

Answer:

0

Question 4

What is the height of a leaf node?

Answer:

1

Question 5

What is the balance factor formula?

Answer:

$BF = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$

Question 6

Which balance factors are valid in an AVL tree?

Answer:

-1, 0, +1

Question 7

What does $BF = +2$ mean?

Answer:

The left side is too tall, so the node is imbalanced.

Question 8

What does $BF = -2$ mean?

Answer:

The right side is too tall, so the node is imbalanced.

Question 9

Why do AVL nodes store height?

Answer:

So the program can calculate balance factor quickly in $O(1)$ time.

Question 10

Should we calculate height from top to bottom or bottom to top?

Answer:

Bottom to top, because parent height depends on child height.

32. Final Phase 1 Summary

An AVL tree is a balanced Binary Search Tree.

A BST follows this rule:

```
left < node < right
```

A normal BST can become tall and slow if values are inserted in a bad order.

An AVL tree prevents this by checking balance at every node.

Height is calculated as:

```
height = 1 + max(left height, right height)
```

In this curriculum:

```
NULL height = 0  
leaf height = 1
```

Balance factor is calculated as:

```
BF = left height - right height
```

A node is valid if its balance factor is:

```
-1, 0, or +1
```

A node is invalid if its balance factor is:

```
+2, -2, +3, -3, etc.
```

The main job of Phase 1 is to help you detect imbalance.

The main job of Phase 2 will be to fix imbalance using rotations.

33. One-Sentence Memory Line

AVL trees are BSTs that use height and balance factor to detect when a node has become too unbalanced.

34. Phase 1 Mastery Checklist

Use this checklist before moving to Phase 2.

You are ready for Phase 2 if you can:

- Explain what a BST is
- Explain why a normal BST can become slow
- Explain what an AVL tree is
- Define height
- Remember that NULL height is 0
- Remember that leaf height is 1
- Use the height formula
- Use the balance factor formula
- Identify valid balance factors
- Identify invalid balance factors
- Trace height from bottom to top
- Find the imbalanced node in a small tree
- Understand why AVL nodes store height
- Understand `getHeight`
- Understand `getBalanceFactor`

If you can do these, you understand Phase 1 well.